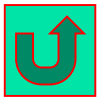
進階編譯命令

Compiler Directives (編譯命令) (1)

- "`include" 編譯命令的功能：目前的 Verilog 電路描述檔可以包含另一個 Verilog 電路描述檔來一起編譯。
- 範例說明:擬設計的模組功能為一個具有4位元的漣波進位加法器 (4 Bits Ripple Carry Adder) 的 Verilog 電路描述檔 (FullAdd4.V)，因此可引用已經設計好的一個具有1位元的全加器之 Verilog 電路描述檔 (FullAdd.V) 來一起編譯合成。
- 程式:

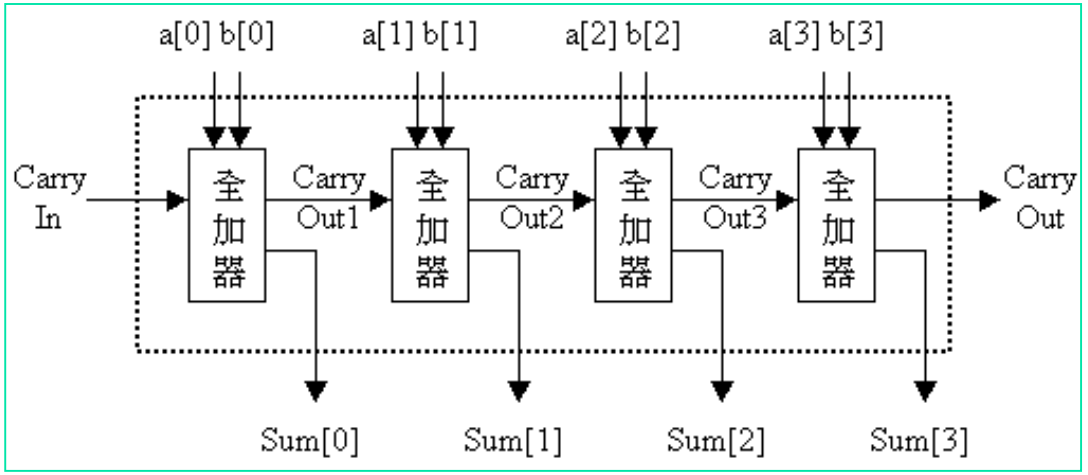
```
// FullAdd4.V, 4位元漣波進位計數器
`include "FullAdd.V"     // 引用FullAdd.V 檔中的所有可用模組

module   FullAdd4 (a, b, Carry_In, Sum, Carry_Out);
input    [3:0] a, b;
input    Carry_In;
output   [3:0] Sum;
output   Carry_Out;
wire     [3:0] Sum;
wire     Carry_Out;
```



"c:\98_test/FullAdd.V" (×)
"c:\\98_test/FullAdd.V"(√)
"c:/98_test/FullAdd.V"(√)
"c://98_test/FullAdd.V"(√)

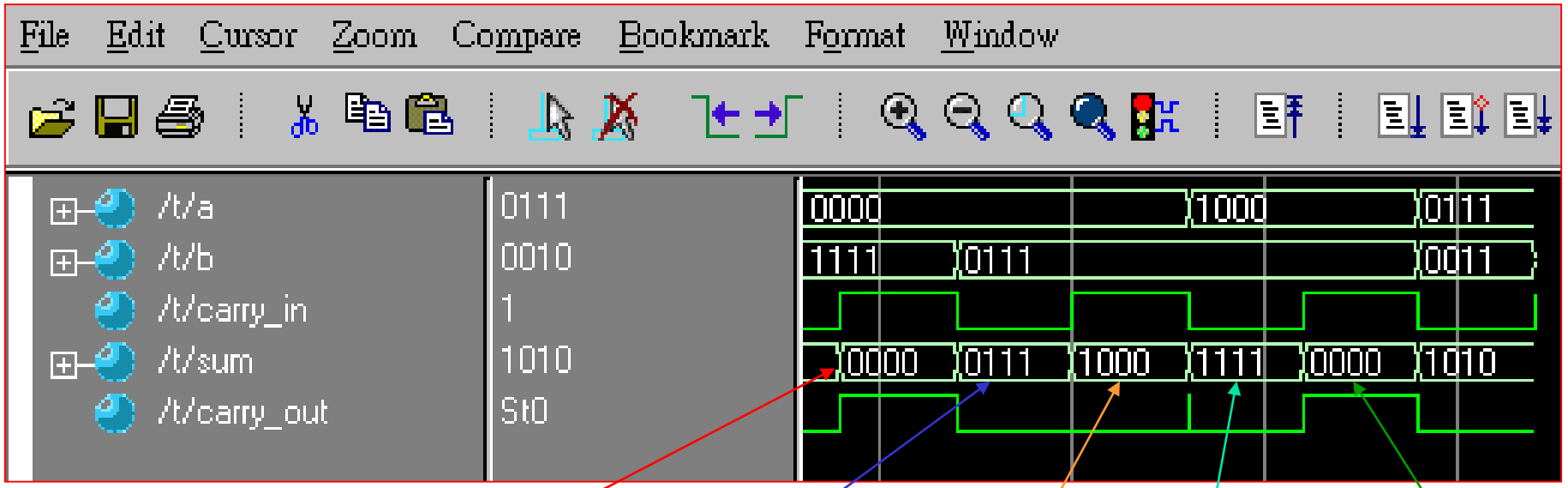
Compiler Directives (編譯命令) (2)

```
// 注意: 底下的 Carry_Out1, Carry_Out2, Carry_Out3 等變數可不宣告
      FullAdd  fa0 (a[0], b[0], Carry_In,   Sum[0], Carry_Out1);
      FullAdd  fa1 (a[1], b[1], Carry_Out1, Sum[1], Carry_Out2);
      FullAdd  fa2 (a[2], b[2], Carry_Out2, Sum[2], Carry_Out3);
      FullAdd  fa3 (a[3], b[3], Carry_Out3, Sum[3], Carry_Out );
endmodule

// FullAdd.V, 1位元的全加器
module  FullAdd (a, b, CarryIn, Sum, CarryOut);
input     a, b, CarryIn;
output    Sum, CarryOut;
wire      Sum, CarryOut;

   assign  {CarryOut, Sum}
    = a + b + CarryIn;
endmodule
```



Simulation Waveform

Core:3_03_FULLADD4



D:\\test\... or D:/test/…

0000	0000	0000	1000	1000
1111	0111	0111	0111	0111
+ 1	+ 0	+ 1	+ 0	+ 1
10000	00111	01000	01111	10000

“`define” Compiler Directives

- `define 編譯命令的功能：類似parameter 敘述。
  我們可以使用 define 編譯命令來表示有限狀態機器的內部狀態以及電路中會用到的常數值。
- 範例1：

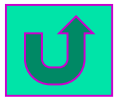
即 RED_Light_Time = 9

```
// 定義：一些在底下會用到的常數值
`define   RED_Light_Time  9   // 紅燈亮的時間倍值

// 定義：有限狀態機器的內部狀態，各個狀態的編碼
parameter  RED_Light    = 2'b00, GREEN_Light = 2'b01,
           YELLOW_Light = 2'b10;
// . . .
   if (Reset)                    // 電路重置
      state = RED_Light;         // 初始狀態設為: RED_Light (紅燈)
   else
      case (state)
        RED_Light:               // 目前狀態為: RED_Light (紅燈)
            if (Count_In == `RED_Light_Time)
               begin             // 重新計數 Counter_16 設為: 1 (重新計數)
               // . . .
               end
// . . .
```

Scalable Design

- 在 Verilog 描述電路中，可以下列**二種方法**來達到**擴充 adder** 這個具有易於調整性的模組。
  - 使用***defparam 敘述去取代 (overriding) parameter 值***。
  - 使用 ***#'敘述去取代 (overriding) parameter 值***。
- 範例:

```
// Scalable.V：建立一個具有易於調整性的 Adder
module     Adder (A, B, Carry_In, Sum, Carry_Out);
parameter  size = 1; // 宣告一個名為 size 的 parameter, 且等於1
input      [size-1:0] A, B;   // 'A' 和  'B' 也使用了名為 size 的參數
input      Carry_In;
output     [size-1:0] Sum; // 'Sum' 使用了名為 size 的這個 parameter
output     Carry_Out;
wire       [size-1:0] Sum;
           // output [size-1:0] Sum; 和 output [1-1:0] Sum; 是相等的
wire       Carry_Out;
    assign  {Carry_Out, Sum} = A + B + Carry_In;
endmodule
```



“defparam” Description

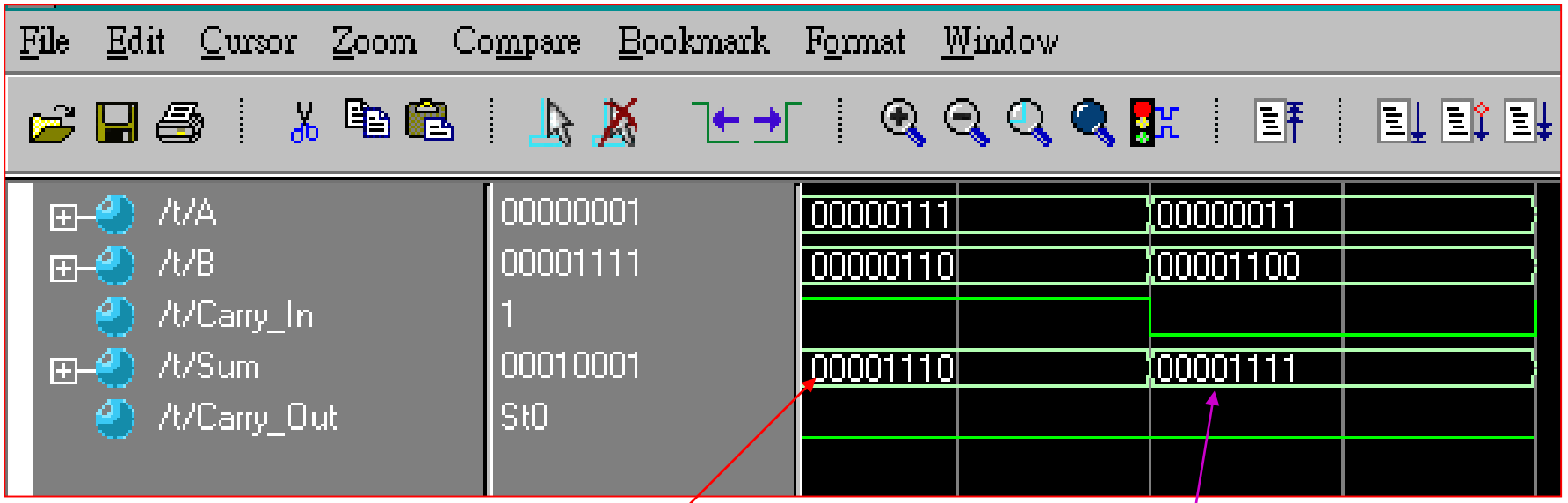
```
// Adder8.V：使用 defparam 敘述去取代(overriding) parameter 值
`include  "Scalable.V"  // 引用 Scalable.V, 一個具有易於調整性的 Adder

module   Adder8 (A, B, Carry_In, Sum, Carry_Out);
parameter  our_size = 8;                  // 定義為 8個位元
input        [our_size-1: 0] A, B;
input        Carry_In;
output       [our_size-1: 0] Sum;
output       Carry_Out;

Adder     My_Adder8 (A, B, Carry_In, Sum, Carry_Out);
  /* defparam My_Adder8.size 將會改寫在 My_Adder8
                  這個 Adder 別名中的 size 值 */
     defparam  My_Adder8.size = 8;    // 定義為 8個位元

endmodule
```

別名

修改數字觀其變化

Simulation Waveform

Core:3_04_ADDER8



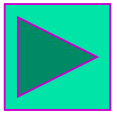

```
  00000111
  00000110
+         1
  00001110
```

```
  00000011
  00001100
+         0
  00001111
```

“#” Description

// Adder16.V：*使用 '#' 敘述去取代 (overriding) parameter 值、*
// 來達到擴充 Adder 這個具有易於調整性的模組

```
`include  "Scalable.V"  //引用 Scalable.V, 一個具有易於調整性的 Adder

module      Adder16 (A, B, Carry_In, Sum, Carry_Out);
parameter   our_size = 16;  // 定義為16個位元
input       [our_size-1: 0] A, B;
input       Carry_In;
output      [our_size-1: 0] Sum;
output      Carry_Out;

/* our_size 將會改寫在 My_Adder16 這個 Adder 別名中的 size 值 */
  Adder  #(our_size)  My_Adder16 (A, B, Carry_In, Sum, Carry_Out);

endmodule
```

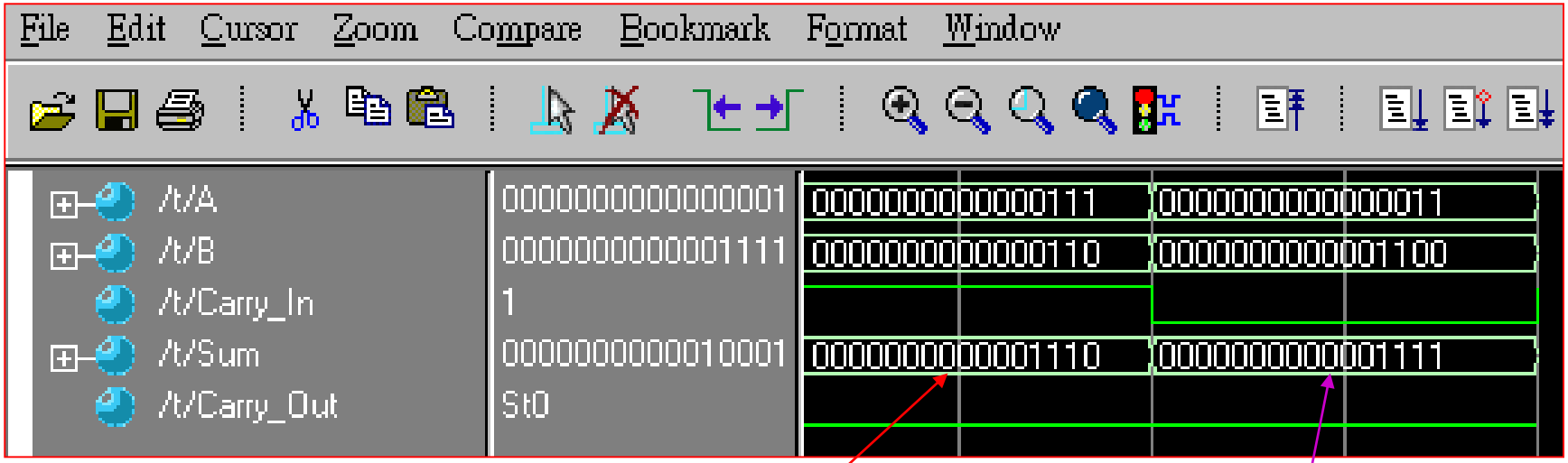


如何修改數字(By order, By name)？

```
Adder  #(our_size, your_size)  My_Adder16 (A, B, Carry_In, Sum, Carry_Out);
```

Simulation Waveform

Core:3_05_ADDER16




```
  0000000000000111
  0000000000000110
+                1
------------------
  0000000000001110
```

```
  0000000000000011
  0000000000001100
+                0
------------------
  0000000000001111
```

New “#” Description (1)

(1).Scalable.v

```
module    Adder(A, B, Carry_In, Sum, Carry_Out);
parameter  size1 = 1;  // «Å§i¤@Ó¦W¬˚ size ªº parameter, ¥Bµ¥©ó1
parameter  size2 = 2;
input      [size1-1:0] A, B;  // 'A' ©M 'B' ¤]¨Ï¥Î¤F¦W¬˚ size ªº³oÓ parameter
input              Carry_In;
output     [size1-1:0] Sum;  // 'Sum' ¨Ï¥Î¤F¦W¬˚ size ªº³oÓ parameter
output             Carry_Out;
wire       [size1-1:0] sum;
//  output  [size-1:0] Sum;  ©M  output  [1-1:0] Sum; ¬O¬Ûµ¥ªº
wire               Carry_Out;
```

New “#” Description (2)

(2).Adder16.v

```
module    Adder16(A, B, Carry_In, Sum, Carry_Out);
parameter  our_size1 = 8;  // ©w¸q¬°16Ó¦ì¤¸
parameter  our_size2 = 16;
input      [our_size1-1: 0] A, B;
input                 Carry_In;
output     [our_size1-1: 0] Sum;
output                Carry_Out;

  /* our_size ±N·|§ï¼g¦b My_Adder16 ³oÓ Adder §O¦W¤¤ªº size È */
  Adder  #(our_size1, our_size2)  My_Adder16 (A, B, Carry_In, Sum, Carry_Out);
endmodule
```

New “#” Description (3)

(3).Adder16_test.v

Core:3_05_ADDER16_1

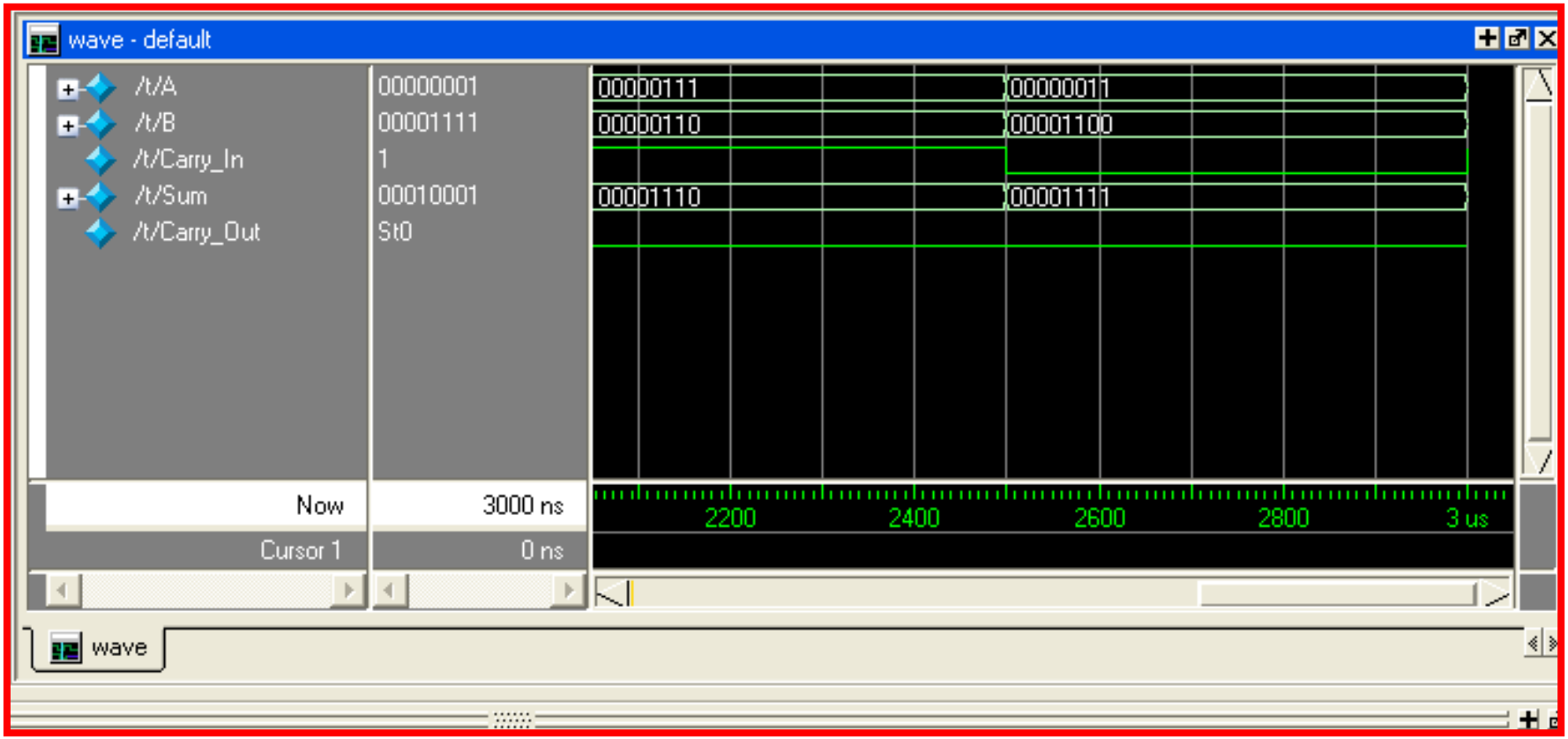
```
module t;
parameter  our_size1 = 8;  // ©w¸q¬ ˚16Ó¦ì¤¸¸
parameter  our_size2 = 16;
reg [our_size1-1: 0] A, B;
reg Carry_In;
wire [our_size1-1: 0] Sum;
wire Carry_Out;

    Adder16 m
      (.A(A), .B(B), .Carry_In(Carry_In), .Sum(Sum), .Carry_Out(Carry_Out));

    // Enter fixture code here
    initial ….
```

```
Adder #(our_size, your_size) My_Adder16 (A, B, Carry_In, Sum, Carry_Out);
```

主程式與副程式的呼叫:一對一, 多對多, 或少對多均可, 但多對少不可以。

Simulation Waveform

Core:3_05_ADDER16_1



Constant—” parameter” Description

- parameter 敘述: 我們可以使用 parameter 敘述來定義：輸出入埠的寬度、向量 (Vectors) 表示法 (變數的位元數)、陣列 (Arrays) 表示法 (陣列的大小)，以及記憶體 (Memory) 表示法。

  - **parameter our_size** = 16; // 定義為16個位元
  - **parameter** Mem_1K = 1024; // 定義一個名字為 Mem_1K 的參數
  - input [**our_size**-1: 0] A, B; // 輸入埠的寬度
  - output [**our_size**-1: 0] Sum; // 輸出埠的寬度

記憶體表示法

陣列 名稱 向量

  - // 向量 (Vectors) 表示法
  - reg Mem1Bit [**Mem_1K**-1:0]; // 1024個1 Bits 的 Register
  - // 陣列 (Arrays) 表示法
  - reg [**our_size**-1:0] Word; // Word 為1個Register, 且擁有16個 Bits

修改

  - // 記憶體 (Memory) 表示法
  - reg [**our_size**-1:0] RegFile [**Mem_1K-1**:0];
  - // 1024個Register, 每個 Register 為16個 Bits

“`define” Compiler Directives (1)

- 我們可以使用 `define 編譯命令來表示：有限狀能機器的內部狀態以及電路中會用到的常數值。
- 範例：

```
reg    [1:0] state;

// 定義：一些在底下會用到的常數值
`define   RED_Light_Time    9      // 紅燈亮的時間倍值
`define   GREEN_Light_Time  6      // 綠燈亮的時間倍值
`define   YELLOW_Light_Time 1  // 黃燈亮的時間倍值

// 定義：有限狀能機器的內部狀態，各個狀態的編碼
parameter  RED_Light = 2'b00, GREEN_Light = 2'b01,
                YELLOW_Light = 2'b10;

always @(posedge Clock)
begin
   if (Reset)      // 電路重置動作
```

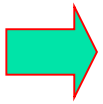
“`define” Compiler Directives (2)

```
if (Reset)      // 電路重置動作
   begin        // 初始狀態設為: RED_Light (紅燈)
     state = RED_Light;
     RED    = 1'b1;      // 紅燈亮
     GREEN  = 1'b0;
     YELLOW = 1'b0;   // 重新計數 Counter_16 設為: 1 (重新計數)
   end
else
begin
   case (state)
            RED_Light:  // 目前狀態為: RED_Light (紅燈)
               begin      // 紅燈: 維持 RED_Light_Time 個時間單位
                  if (Count_In == `RED_Light_Time)
                  begin   // 重新計數 Counter_16 設為: 1 (重新計數)
                  …
                  end
      .
      . 略
      .
    endcase
end
```

“timescale” Description

- “timescale”敘述用於定義模組中的模擬時間單位及精準度。
- 語法格式:
  ‘timescale <時間單位>/<時間精度>
- 範例:

```
‘timescale 10ns/1ns
module time_scale;
      reg      time_set;
      parameter p=1.65;
      Initial
         begin
            #p time_set=0;
            #p time_set=1;
         end
endmodule
```

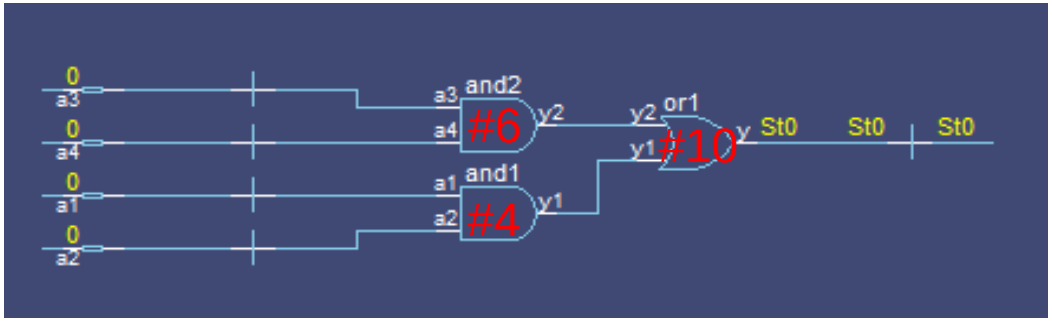


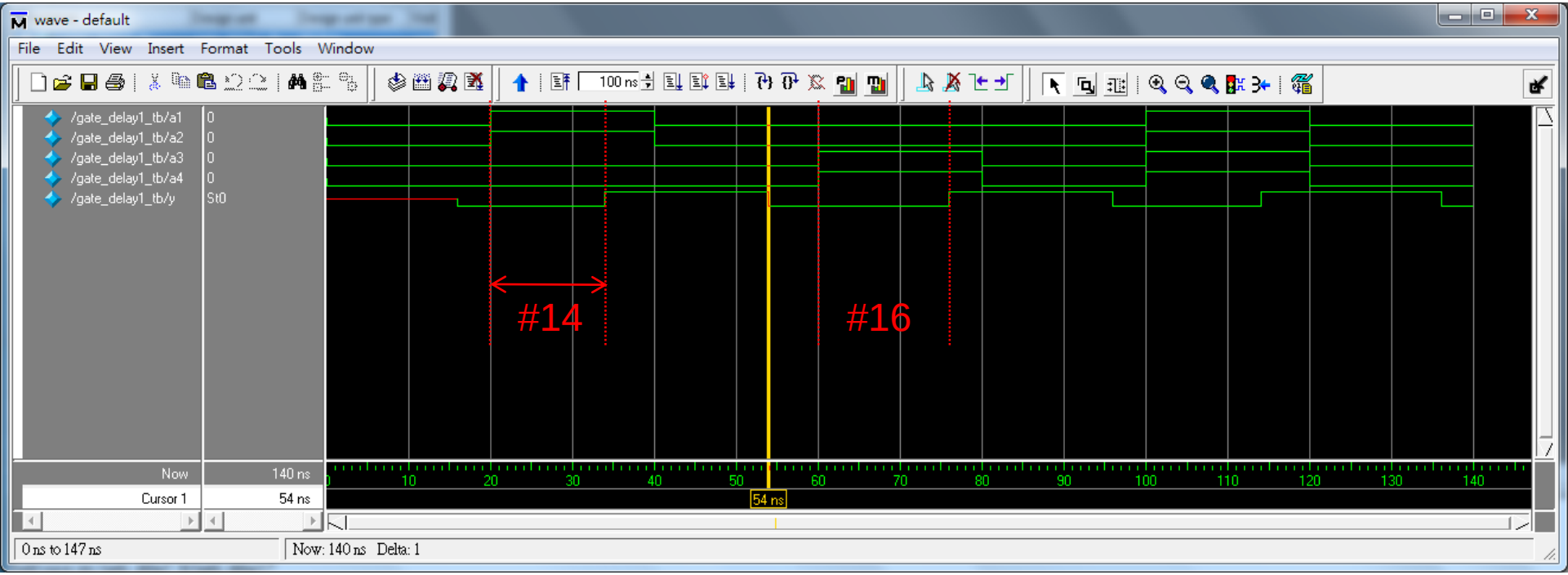
```
0    time_set=X
17   time_set=0
      //(P=1.65 ≈ 17ns)
34   time_set=1
      //(P+P≈17+17=34ns)
```

時序延遲設定

Gate Delay

- Gate delay:邏輯閘延遲可分為:針對電路中個別邏輯閘或輸出端的邏輯閘設定其延遲參數。





範例練習

Gate Delay(7-001)

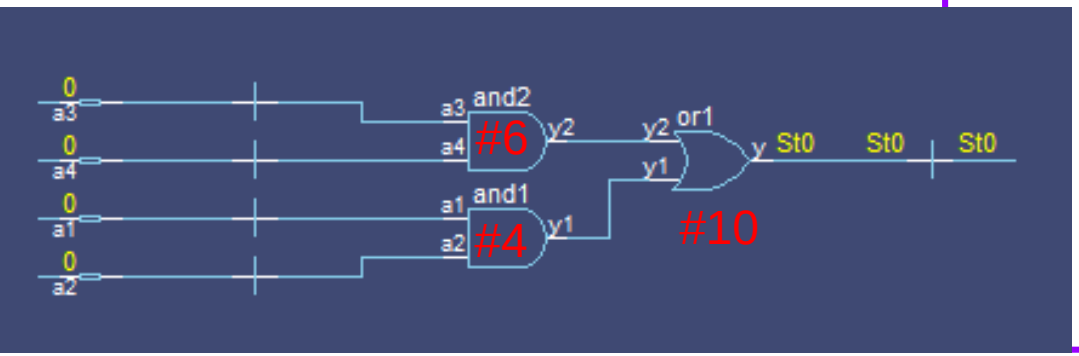
以邏輯閘層描述方式

設定個別邏輯閘的延遲

```
//Gate delay for individual gate using
  gate-level modeling


module gate_delay1(a1, a2, a3, a4, y);
    input a1, a2, a3, a4;
    output y;
    wire y1, y2;
    and #4 and1(y1, a1, a2);
    and #6 and2(y2, a3, a4);
    or #10 or1(y, y1, y2);
endmodule
```

Test bench

```
module gate_delay1_tb();
    reg a1, a2, a3, a4;
    wire y;

    gate_delay1 gate_delay1(.a1(a1),
                            .a2(a2),
                            .a3(a3),
                            .a4(a4),
                            .y(y)
                            );

    initial
        begin
            #0  a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;
            #20 $stop;
        end
endmodule
```



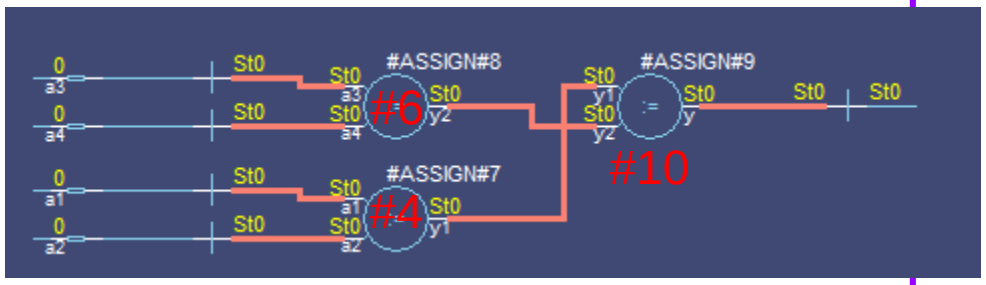
範例練習

Gate Delay(7-002)

以資料流描述方式設定

個別邏輯閘的延遲

```
//Gate delay for individual gate using
 data-flow modeling


module dataflow_delay1(a1, a2, a3, a4,
 y);
    input a1, a2, a3, a4;
    output y;
    wire y1, y2;
    assign #4 y1 = a1 & a2;
    assign #6 y2 = a3 & a4;
    assign #10 y = y1 | y2;
endmodule
```



Test bench

```
module dataflow_delay1_tb();
    reg a1, a2, a3, a4;
    wire y;
    dataflow_delay1
dataflow_delay1(.a1(a1),
                .a2(a2),
                .a3(a3),
                .a4(a4),
                .y(y)
                );

    initial
        begin
            #0  a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;
            #20 $stop;
        end
endmodule
```

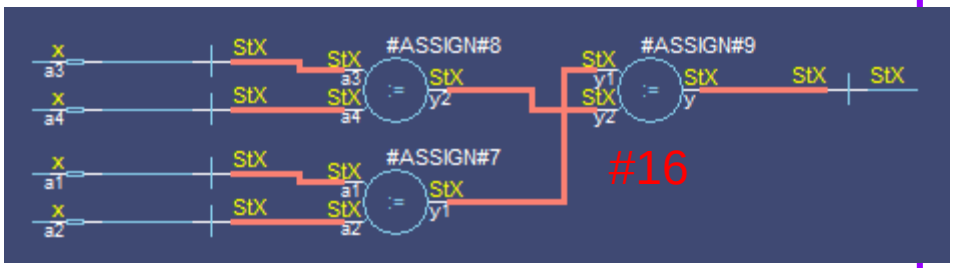
範例練習

Gate Delay(7-003)

以資料流描述方式設定

輸出端邏輯閘的延遲

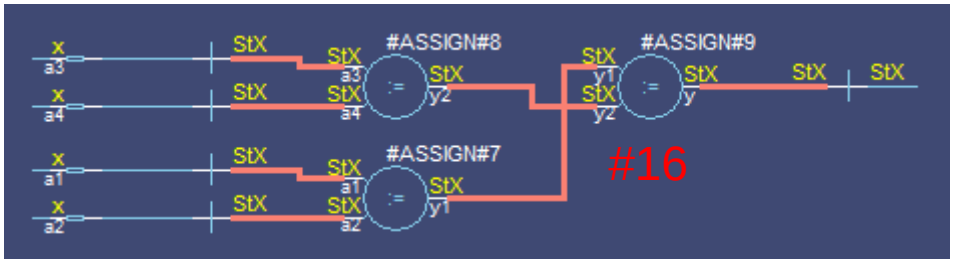
```
//Gate delay for the last gate using
 data-flow modeling


module dataflow_delay2(a1, a2, a3, a4,
 y);
    input a1, a2, a3, a4;
    output y;
    wire y1, y2;
    assign y1 = a1 & a2;
    assign y2 = a3 & a4;
    assign #16 y = y1 | y2;
endmodule
```

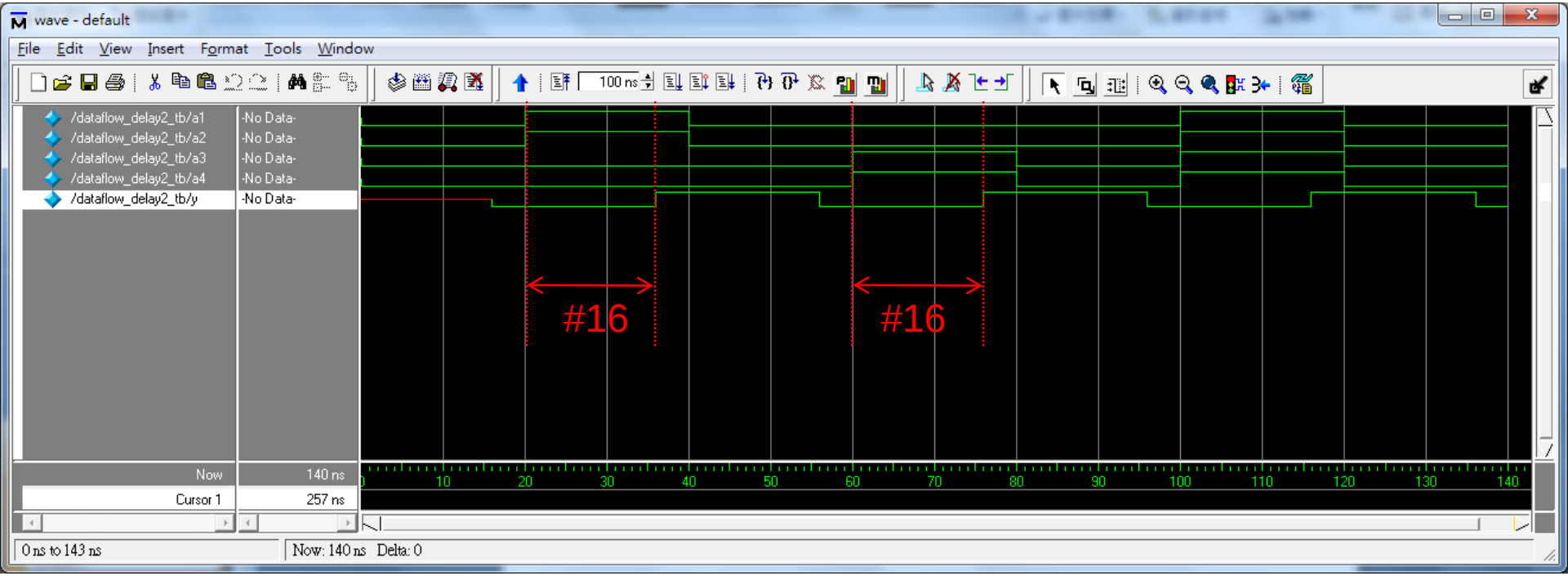


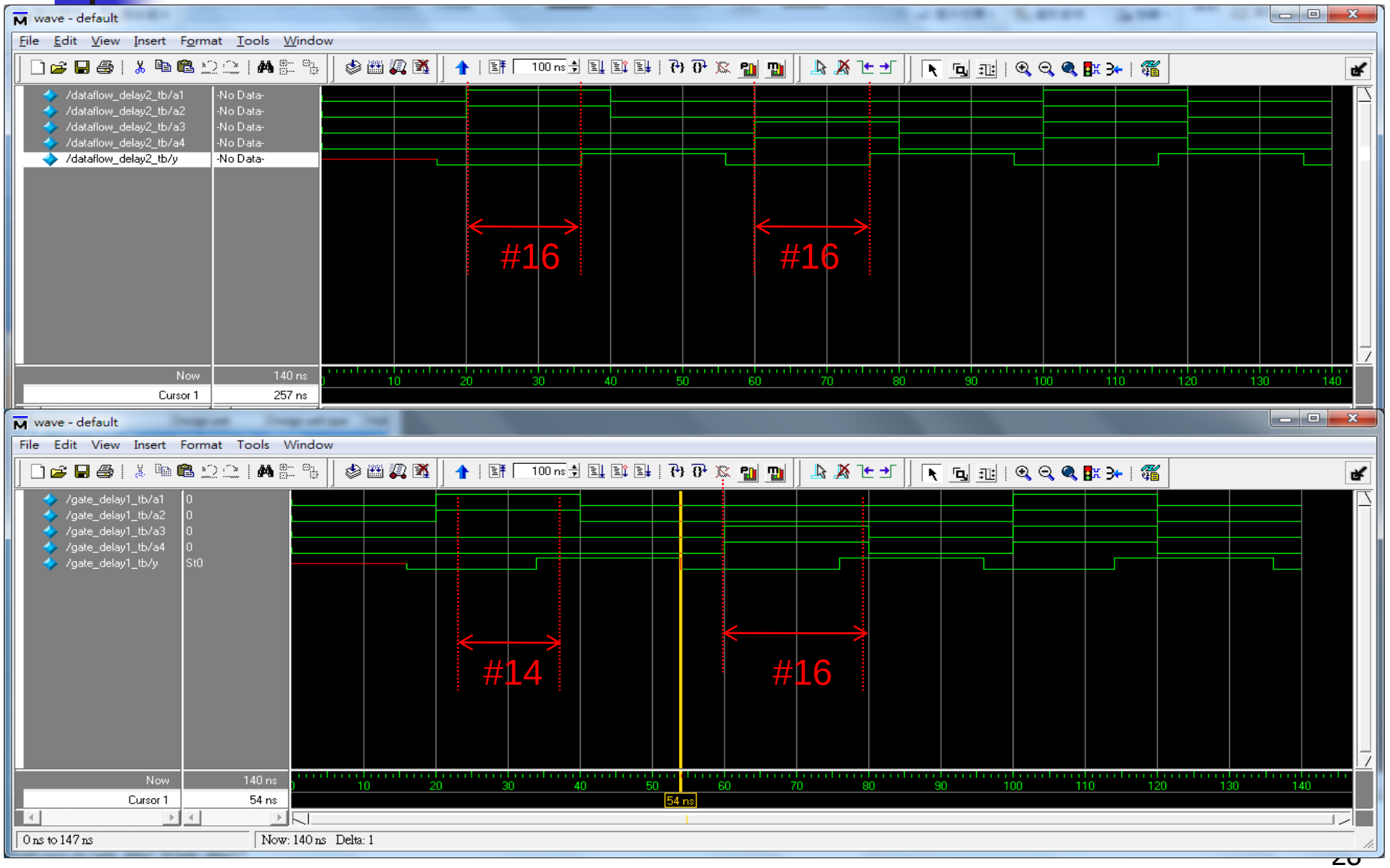
Test bench

```
module dataflow_delay2_tb();
    reg a1, a2, a3, a4;
    wire y;
    dataflow_delay2
dataflow_delay2(.a1(a1),
                .a2(a2),
                .a3(a3),
                .a4(a4),
                .y(y)
                );

    initial
        begin
            #0  a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;
            #20 $stop;
        end
endmodule
```

Simulation Waveform

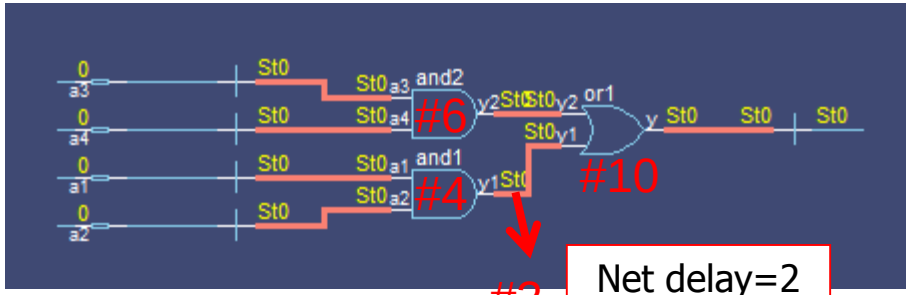


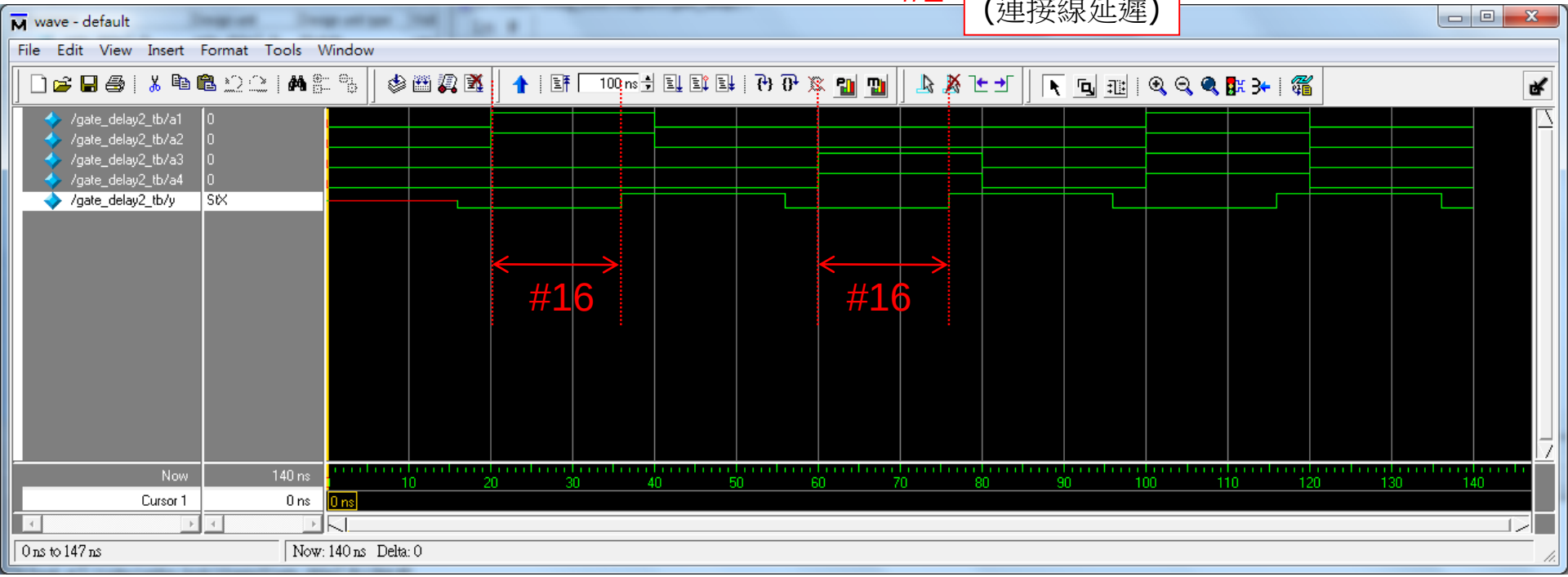



wave - default
File Edit View Insert Format Tools Window
/dataflow_delay2_tb/a1
/dataflow_delay2_tb/a2
/dataflow_delay2_tb/a3
/dataflow_delay2_tb/a4
/dataflow_delay2_tb/y
-No Data-
#16
#16
Now
140 ns
Cursor 1
257 ns
wave - default
File Edit View Insert Format Tools Window
/gate_delay1_tb/a1
/gate_delay1_tb/a2
/gate_delay1_tb/a3
/gate_delay1_tb/a4
/gate_delay1_tb/y
St0
#14
#16
Now
140 ns
Cursor 1
54 ns
0 ns to 147 ns
Now: 140 ns Delta: 1

Net Delay

- Net delay:連接線的延遲在於設定一邏輯閘之輸出到另一邏輯閘的輸入之間連接的時間延遲。





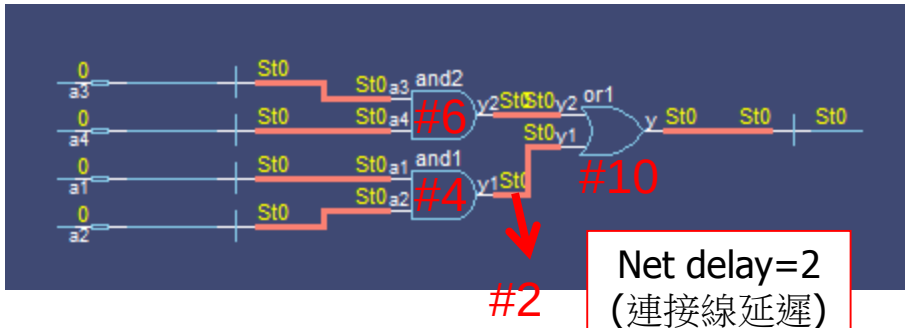
範例練習

Net Delay(7-004)

以邏輯閘層描述方式

設定連接線信號轉換延遲

```
//Net delay using gate-level moduling

module gate_delay2(a1, a2, a3, a4, y);
    input a1, a2, a3, a4;
    output y;
    wire #2 y1;
    wire y2;
    and #4 and1(y1, a1, a2);
    and #6 and2(y2, a3, a4);
    or #10 or1(y, y1, y2);
endmodule
```



Test bench

```
module gate_delay2_tb();
    reg a1, a2, a3, a4;
    wire y;
    gate_delay2 gate_delay2(.a1(a1),
                            .a2(a2),
                            .a3(a3),
                            .a4(a4),
                            .y(y)
                            );
    initial
        begin
            #0  a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 0;
                a2 = 0;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;

            #20 a1 = 1;
                a2 = 1;
                a3 = 1;
                a4 = 1;

            #20 a1 = 0;
                a2 = 0;
                a3 = 0;
                a4 = 0;
            #20 $stop;
        end
endmodule
```

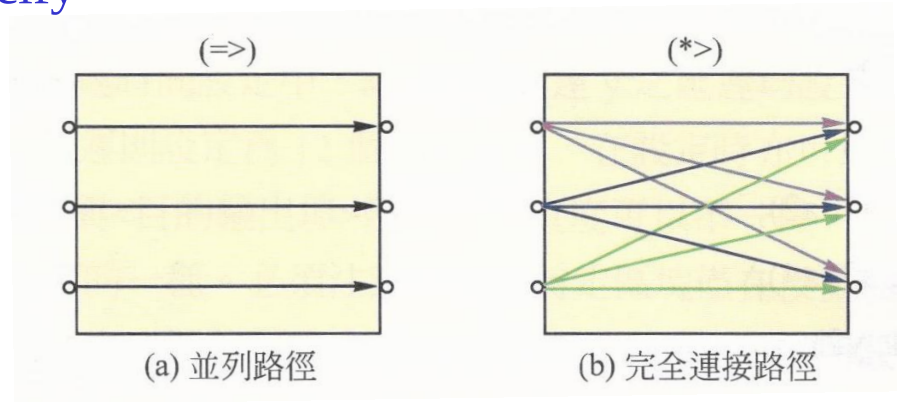
Module Path Delay

- 輸出入埠之路徑延遲時間的設定(並列路徑)格式如下:

```
specify
    <來源輸入埠1>=><目的輸出埠1>=<延遲時間1>;
    <來源輸入埠2>=><目的輸出埠2>=<延遲時間2>;
      …
    <來源輸入埠n>=><目的輸出埠n>=<延遲時間n>;
endspecify
```

- 完全連接路徑延遲時間的設定(完全連接路徑)格式如下:

```
specify
    <來源輸入埠1>*><目的輸出埠1>=<延遲時間1>;
    <來源輸入埠2>*><目的輸出埠2>=<延遲時間2>;
      …
    <來源輸入埠n>*><目的輸出埠n>=<延遲時間n>;
endspecify
```

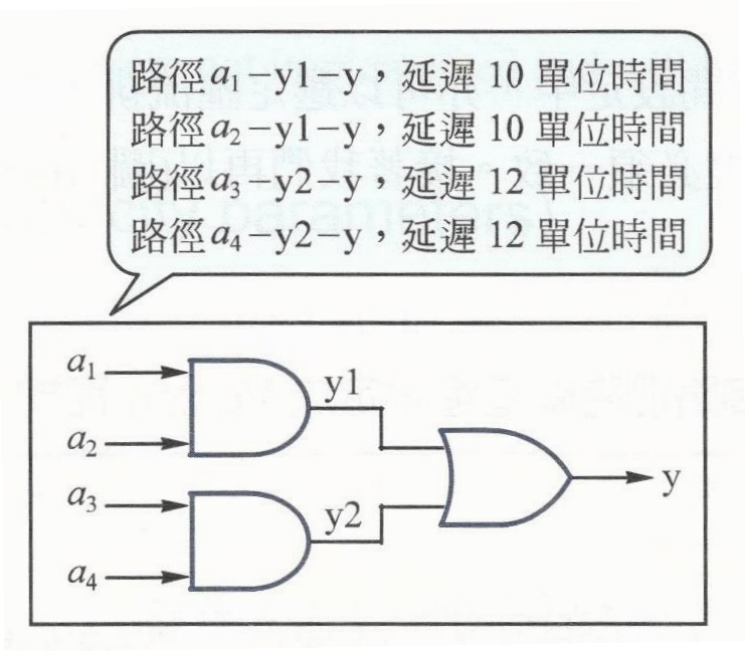


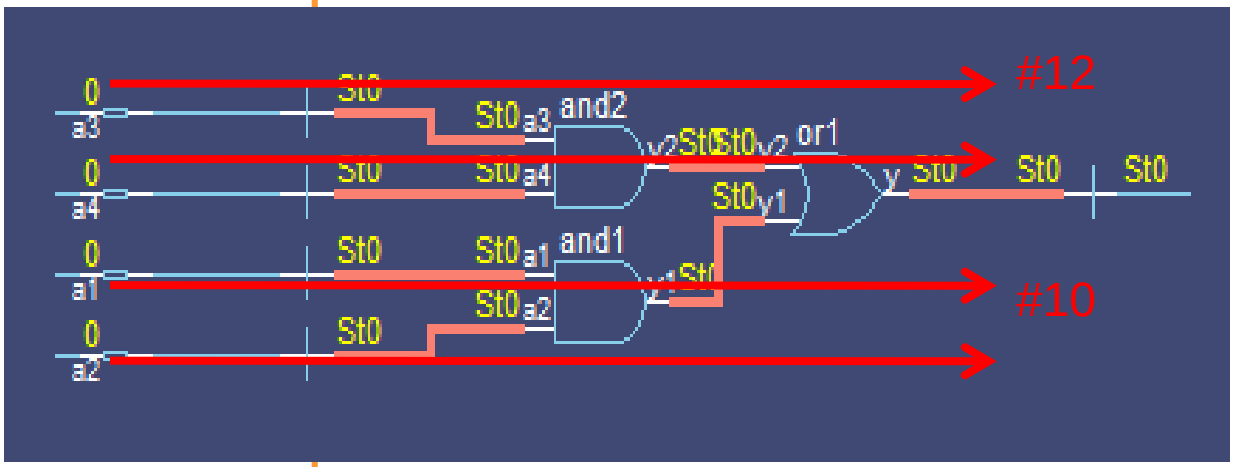
範例練習

Parallel Path Delay(7-005)

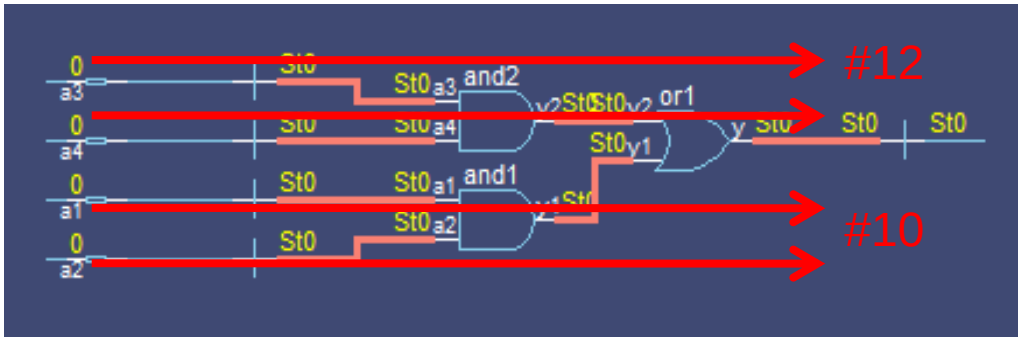
並列路徑時間延遲的設定

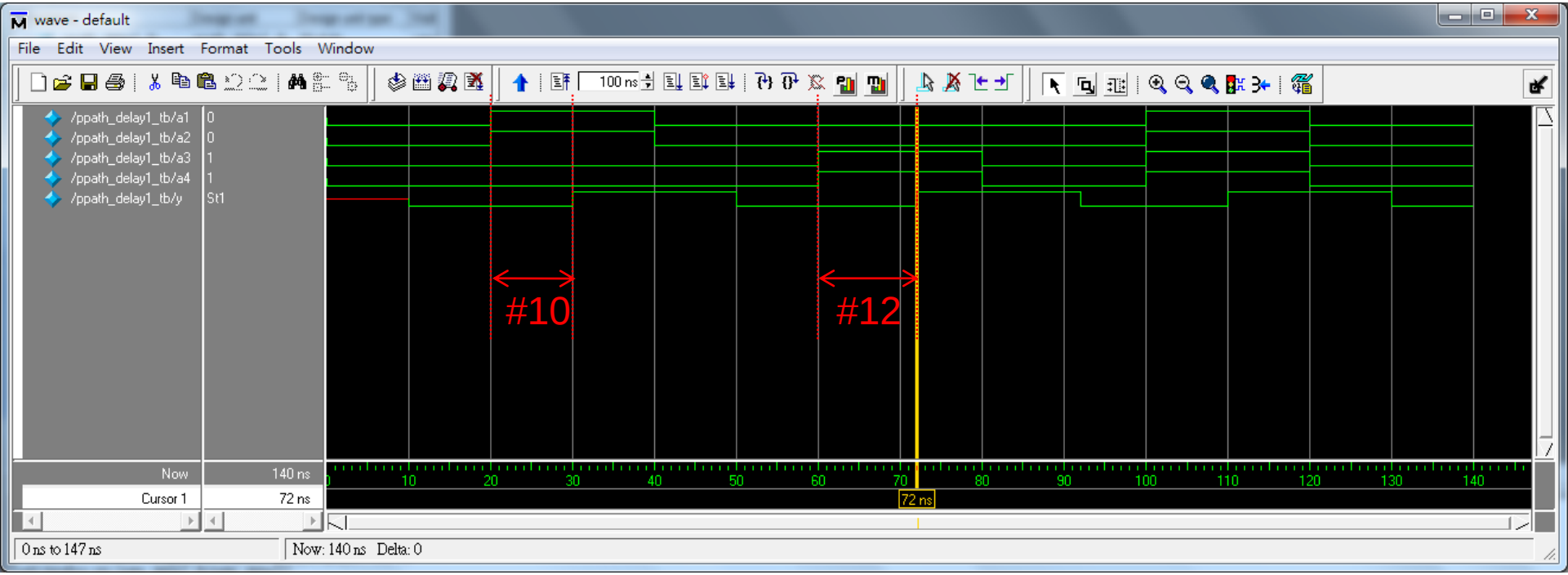
```
//Parallel path delay using gate-level
  modeling
module ppath_delay1(a1, a2, a3, a4, y);
    input a1, a2, a3, a4;
    output y;
    wire y1, y2;

    specify
       (a1 => y) = 10;
       (a2 => y) = 10;
       (a3 => y) = 12;
       (a4 => y) = 12;
    endspecify

    and and1(y1, a1, a2);
    and and2(y2, a3, a4);
    or or1(y, y1, y2);
endmodule
```





Simulation Waveform



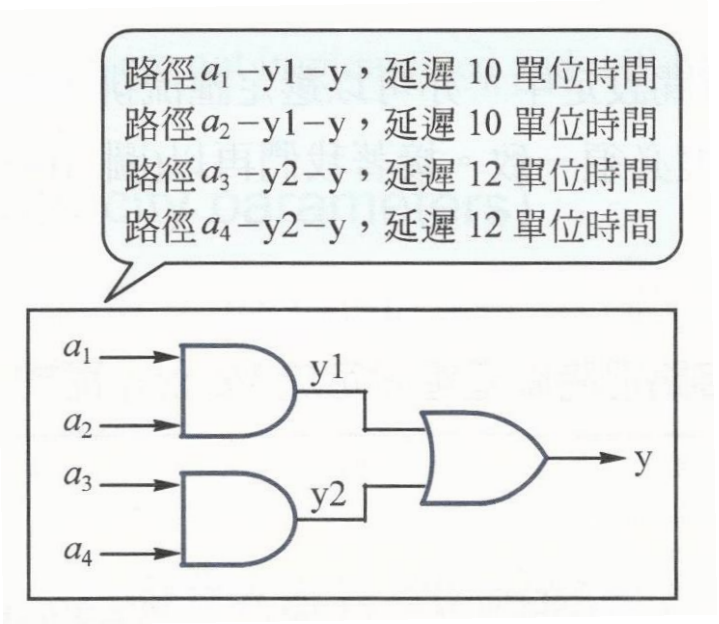


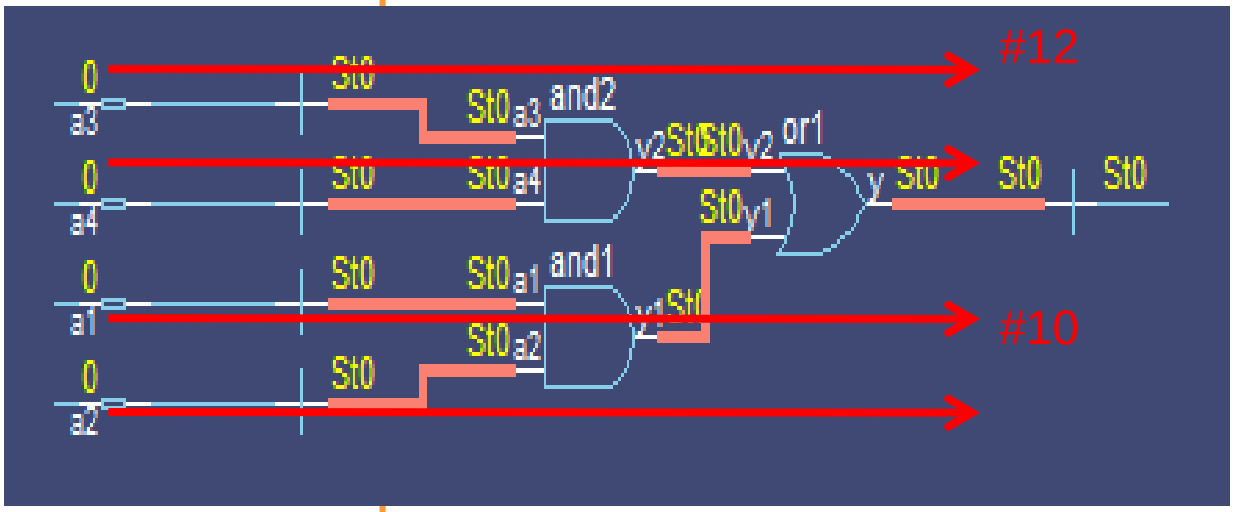
範例練習

Full Connection Path Delay(7-006)

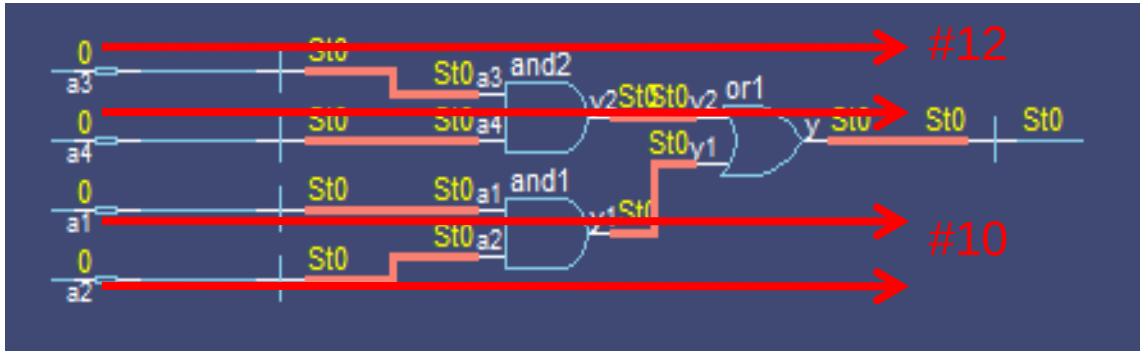
完全連接路徑時間延遲的設定

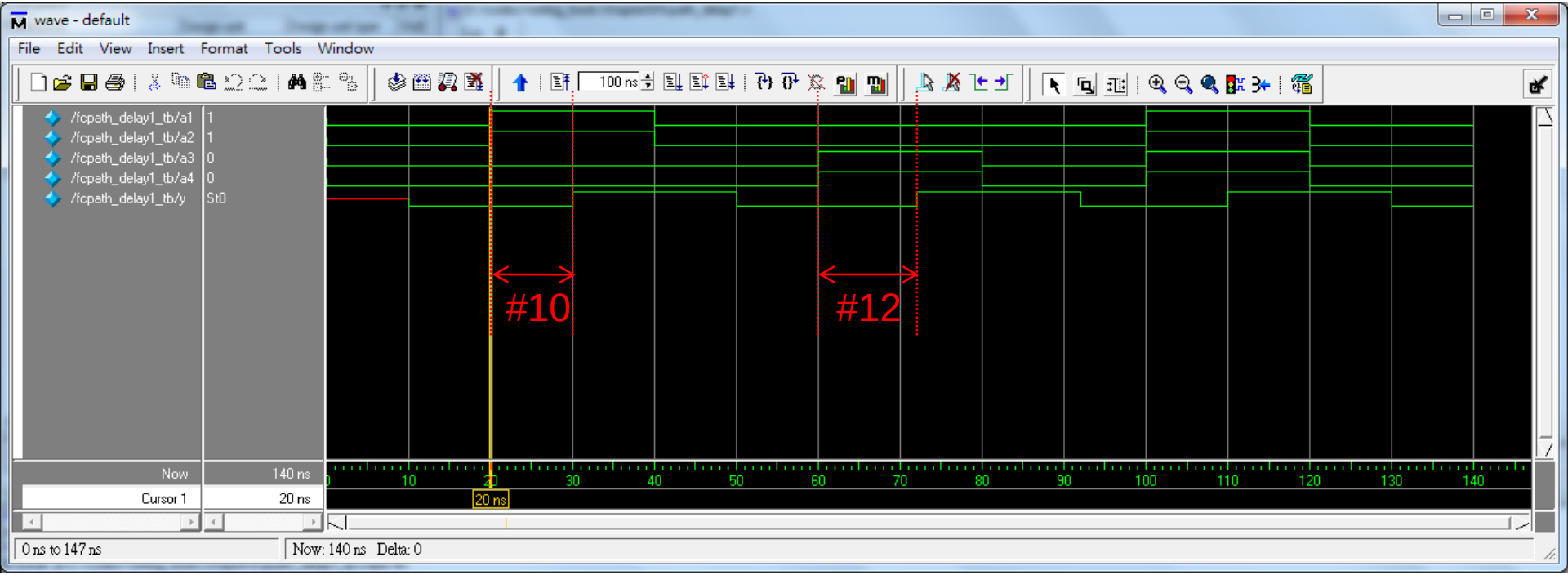
```
//Full connection path delay using gate-
  level modeling

module fcpath_delay1(a1, a2, a3, a4, y);
    input a1, a2, a3, a4;
    output y;
    wire y1, y2;

    specify
        (a1, a2 *> y) = 10;
        (a3, a4 *> y) = 12;
    endspecify

    and and1(y1, a1, a2);
    and and2(y2, a3, a4);
    or or1(y, y1, y2);
endmodule
```





Simulation Waveform

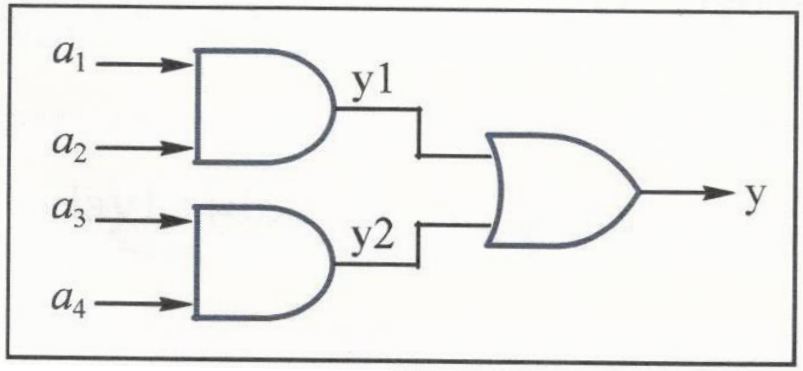




State-Dependent Path Delay

- **狀態相關路徑延遲(條件式延遲)**:若條件成立時，則設定延遲時間；否則，則不設定延遲時間。
- 其語法格式依並列或完全連接路徑而分為；
  - If (<條件判斷式)<來源輸入埠>=><目的輸出埠>=<延遲時間>;
  - If (<條件判斷式)<來源輸入埠>*><目的輸出埠>=<延遲時間>;

1. 若$(a_1\ a_2)$=(11)則
   路徑 a_1−y1−y 與 a_2−y1−y，均延遲 10 單位時間
2. 若$(a_3\ a_4)$=(11)則
   路徑 a_3−y2−y 與 a_4−y2−y，均延遲 12 單位時間

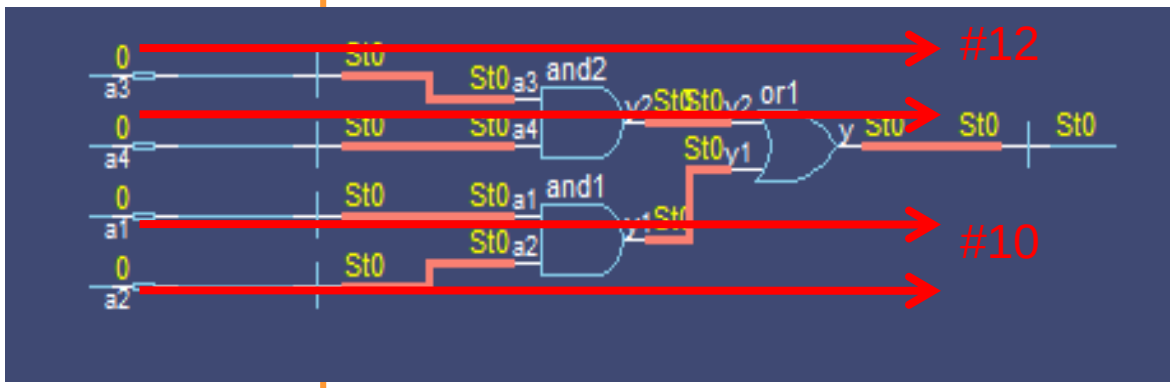


範例練習

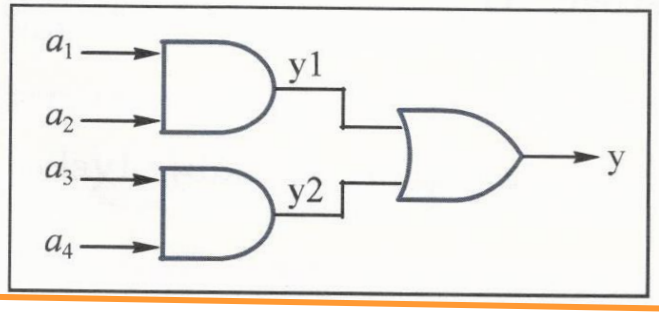
Conditionally Full Connection Path Delay(7-009)

條件式完全連接路徑時間延遲的設定

```
//Conditionally full connection path delay
  using gate-level modeling
//ex10.10
module cfcpath_delay(a1, a2, a3, a4, y);
    input a1, a2, a3, a4;
    output y;
    wire y1, y2;

    specify
        if ({a1, a2} ==  2'b11)
            (a1, a2 *> y) = 10;
        if ({a3, a4} ==  2'b11)
            (a3, a4 *> y) = 12;
    endspecify

    and and1(y1, a1, a2);
    and and2(y2, a3, a4);
    or or1(y, y1, y2);
endmodule
```



1. 若$(a_1\ a_2)$=(11)則
   路徑 a_1−y1−y 與 a_2−y1−y，均延遲 10 單位時間
2. 若$(a_3\ a_4)$=(11)則
   路徑 a_3−y2−y 與 a_4−y2−y，均延遲 12 單位時間



Simulation Waveform